

pcap が S3 に現れない、を仕組みから理解する

Arkime の全体構造、S3 writer がマルチパートアップロードをどう使っているか、そして「エラーは無いのにアップロードが止まる」が仕様どおりに起こりうる理由をまとめる。

01

Arkime とは何をしているのか

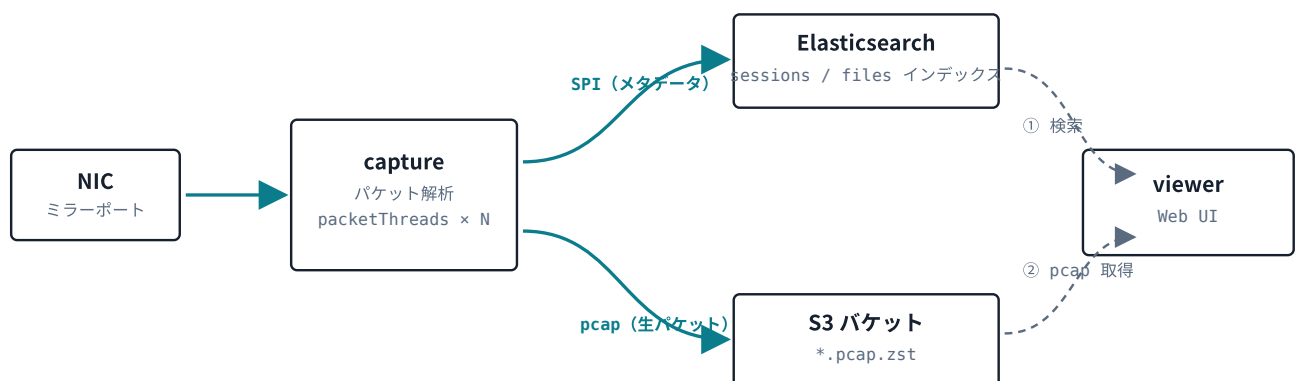
Arkime (旧 Moloch) は、ネットワークを流れるパケットを**全部保存して、後から検索できるようにするシステム**である。構成要素は3つだけ。

- **capture** — C 言語の常駐プロセス。NIC からパケットを読み、解析し、書き出す
- **Elasticsearch / OpenSearch** — 「いつ・誰が・誰と・何のプロトコルで通信したか」というメタデータ (SPI) の置き場
- **viewer** — Node.js の Web UI。検索して、必要なら生パケットを取り出す

ここが一番大事：データは2つに分かれて流れる

Arkime は **メタデータ**と**生パケット**を別々の場所に置く。メタデータは Elasticsearch へ、生パケット (pcap) はローカルディスクか S3 へ。この2本は独立して動く。

だから「**viewer でセッションは見えるのに、pcap をダウンロードしようとすると出てこない**」という状態が普通に起こりうる。今回の事象はまさにこれである。



この2本は独立。上が動いていても下が止まることもある。

capture は1つのパケットから2種類の出力を作る。viewer は ES で当たりを付け、pcap は別途 S3 から取りに行く。

ES 側の `arkime_files` インデックスが、この 2 本をつなぐ索引になっている。「ファイル番号 N の実体はここにあり、サイズはこれだけ」という台帳で、**ファイルを作った瞬間に登録され、ファイルを閉じた瞬間にサイズが埋まる**。閉じられていないファイルは、台帳上サイズ 0 のまま残る。

02

S3 writer はなぜ「閉じるまで何も出てこない」のか

ローカルディスクなら、capture はファイルを開いて追記していけばいい。途中でプロセスが死んでも、そこまでのファイルは残る。

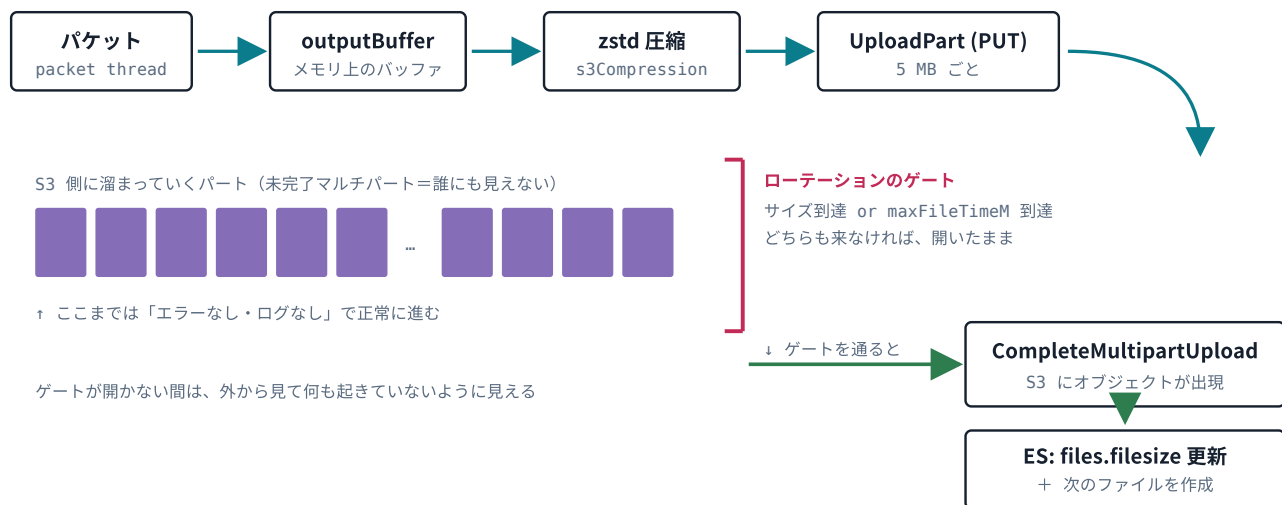
S3 にはこの「追記」が無い。オブジェクトは丸ごと 1 回で置くものなので、数 GB のファイルを少しずつ送るには **マルチパートアップロード** という 3 段構えの API を使う。

段階	やること
CreateMultipartUpload	「これから分割アップロードを始める」と宣言し、 <code>uploadId</code> を受け取る
UploadPart × N	5 MB 以上のかたまりを何度も PUT する。パートは S3 側に溜まるが、まだオブジェクトではない
CompleteMultipartUpload	「全部送った、つなげてくれ」と伝える。ここで初めてオブジェクトが出現する

Complete を打つまで、S3 にファイルは「存在しない」

途中のパートは確かに S3 に届いていて、ストレージ料金も発生している。しかし `ls` しても出てこないし、ダウンロードもできない。見えるのは `list-multipart-uploads` (未完了アップロード一覧) だけである。

そして Arkime が Complete を打つのは、**ファイルをローテーションするときだけ**。逆に言えば、ローテーションが起きない限り S3 には 1 つもオブジェクトが現れない。



パケットは順調に流れ、パートも順調に S3 に届いている。それでもゲートが開かなければ、観測できる成果物はゼロになる。

ゲートが開く条件は 2 つしかない

契機	設定	既定の挙動
サイズ到達	maxFileSizeG pcapWriteSize	実効しきい値は <code>min(maxFileSizeG, pcapWriteSize × 1000)</code> 。 <code>pcapWriteSize</code> は S3 では 5 MB 未満に落とせないため、既定なら約 4.88 GiB 。しかもこれは 圧縮後の サイズで比較される
時間到達	maxFileTimeM	既定値 0 = 無効 。0 だとタイマーそのものが登録されない

ここが今回の事象の核心

`maxFileTimeM` が未設定なら、契機は**サイズだけ**になる。zstd で 4 倍に圧縮されるなら、1 スレッドあたり生パケットで約 19.5 GB 溜まらなないとゲートは開かない。

`packetThreads=4` のセグメントで換算すると — 10 Mbps で約 17 時間、1 Mbps で約 7 日、100 Kbps で約 70 日。トラフィックが細ったら、S3 に何も出てこないのは正常動作である。

03

今回の事象を、この構造に当てはめる

確定している観測

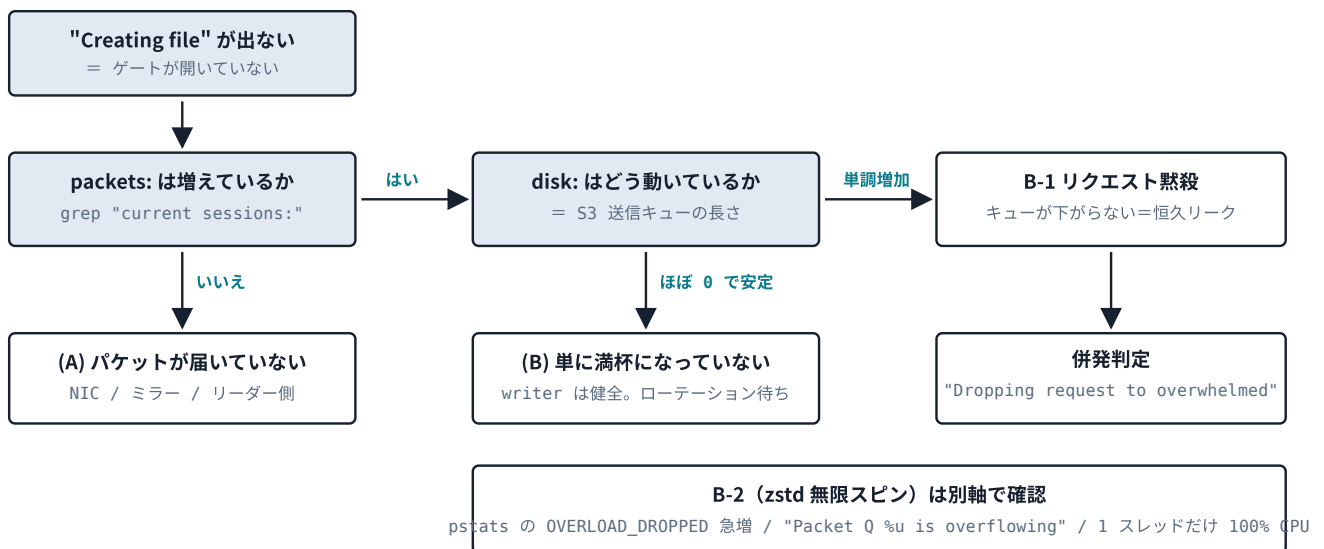
- ある時点 T 以降、pcap が S3 に現れない

- capture.log に目立ったエラーが無い
- T 以降 "Creating file ..." が出ない = ファイルが1度も新規作成されていない
- capture プロセスの再起動なし・ホストもバックエンドも逼迫なし・設定変更なし

「ファイルが新規作成されない」は、02 のゲートの言い換えである。ゲートを通らない限り次のファイルは作られないので、この2つは同じ事実を別の角度から見ているにすぎない。

T は「事故」ではなく「最後の1回のローテーション」

再起動が無い以上、T で作られたファイルは同一プロセス内でずっと開いたままであり、一度も満杯になっていない。T は何かが壊れた瞬間ではなく、最後に正常にゲートを通った瞬間だと読むのが自然である。



capture.log だけで (A) / (B) / B-1 は分かれる。判定に必要な行は定期統計 `current sessions:` の 1 行に全部入っている。

この表だけでは (B) を確定できない

`packets:` が増えていて `disk:` が 0 で安定、という観測は (B) の署名だが、「writer がそもそも呼ばれていない」場合とまったく同じ見え方になる。ルールや `dontSaveBPFs` でセッションの `stopSaving` が 0 になると、パケットは受信・解析されカウントもされるが、writer には 1 つも渡らない。

切り分けには `pstats:` の 1 番目（実処理数）を併せて見る。`packets:` が伸びているのに 1 番目が伸びていなければ、パケットは writer より手前で落ちている。

debug=1 を入れると disk: が 3 つに分解される

`disk:` は「送信待ちバッファ + HTTP キュー + 応答待ち」の合計値で、そのままでは B-1 のどの経路かが分からない。デバッグを有効にすると内訳が出る。

```
queue length: http Q:<N> in progress:<N> waiting:<N>
```

`waiting` が増え続ける = `uploadId` が取れておらず全部バッファに溜まっている (Init 破棄)。`in progress` が高止まり = 送ったのに応答が返ってこない (PUT 破棄)。**B-1 の 2 経路がここで直接分かれる。**

S3 側から見ると、もっと速く決まる

本番にアクセスできるなら、未完了マルチパートのパート数を数えるのが最短経路になる。

```
# 未完了アップロードの一覧と、その開始時刻
aws s3api list-multipart-uploads --bucket <BUCKET>

# 上で得た Key / UploadId で、パートが何番まで積まれているか
aws s3api list-parts --bucket <BUCKET> --key <KEY> --upload-id <ID> \
  --query 'length(Parts)'
```

観測	結論
Initiated が T と一致し、パート数が今も増えている	(B) 確定。writer は健全で、ゲート待ちなだけ
Initiated が T と一致し、パート数が数個で止まっている	B-1 の疑い濃厚。PUT が届いていない
未完了アップロードがそもそも 1 件も無い	Init 自体が成立していない。B-1 の Init 破棄パス、または (A)

ついでに確認しておく価値がある副作用

未完了マルチパートは、完了しないままストレージ料金が発生し続ける。長期に取り残されているものがある場合は、バケットのライフサイクルに `AbortIncompleteMultipartUpload` を入れておく。

B-1 は実在する。上流で修正済み、ただし v6 で

引き継ぎ資料が「潜在バグ」として挙げていた B-1 — HTTP リクエストがコールバックを呼ばずに破棄され、`uploadId` が永久に NULL のままバッファが溜まり続ける — は、推測ではなく上流で認識され、修正された実在のバグである。

Arkime v6.6.0 (2026-07-09) PR #4100

writer-s3 の修正として、パート失敗後に壊れた `CompleteMultipartUpload` を組み立てる問題（現在は `upload` を `abort` する）と並んで、「**multipart init が失敗したときにキューされたバッファが永久にリークする**」問題が挙げられている。B-1 の第 1 の帰結と同じものである。

ただし Arkime 5 には来ない

メンテナは別件のやり取りで、Arkime 5 では修正を行わない方針を明言している。**v5.8.0 に留まる限り、B-1 は環境に残り続けると前提を置く必要がある。**

とはいえ、B-1 は「S3 に上がらない」の説明にはなるが、「`Creating file` が出ない」の説明にはならない。ローテーション判定はキューの状態を見ていないからである。**B-1 は主犯ではなく、併発している可能性のある共犯**という位置づけは、引き継ぎ資料の整理どおりで正しい。

05

対処案と、その落とし穴

提案されている `maxFileTimeM = 60` は方向として正しい。トラフィック量に依存しないローテーション契機を作る唯一の手段であり、空ファイルの量産も内部条件で防がれている。

検証済み：パケットが 1 つも来なくても発火する

タイマーはメインスレッドで 30 秒ごとに動き、各パケットスレッドへコマンドを投げる（`arkime_session_add_cmd_thread`）。受け側のパケットスレッドは、キューが空のとき **1 秒でタイムアウトする条件待ち**に入り、起きたら必ず `arkime_session_process_commands()` を通る。パケットの有無は条件になっていない。

公式リポジトリには「パケットが来るまで時間チェックが走らない」という v5 の報告があるが、報告者の環境は `tcpdump | capture -r -` の標準入力経由という特殊構成で、最終的に `tcpdump` をやめて解消している。**ライブラリー構成であれば、仮説 (A) が当たっていてもタイマーは効く。**

ただし v5 にはタイマー発火のログが無い

時間契機でファイルを閉じたことを示すデバッグ行（`Closing file ... due to time Ns`）は v6 で追加されたもので、v5.8.0 には存在しない。v5 では「タイマーが動いたか」をログから直接は確認できないので、設定投入後は S3 側にオブジェクトが出現するかで検証することになる。

投入するもの

```
[default]
maxFileTimeM = 60
```

あわせて、バケットのライフサイクルに `AbortIncompleteMultipartUpload` を設定し、取り残された未完了マルチパートを掃除する。

投入前に押さえておきたい 1 点

もし原因が (B)（単にゲートに届いていない）なら、この設定は「壊れていたものを直す」のではなく「ファイル粒度を細かくする」変更になる。ファイル数が増えるぶん `arkime_files` のドキュメント数と S3 のオブジェクト数が増えるので、保持ポリシーとの兼ね合いは事前に見ておく。

06

次に集める情報

項目	なぜ必要か
packetThreads	1 なら B-2 が単独で全症状を説明できる。2 以上なら全スレッド共通のグローバル要因に絞られる
current sessions: の推移	<code>packets:</code> と <code>disk:</code> の 2 つで (A)/(B)/B-1 が分かれる
T 以前の "Creating file" 間隔	もともと数時間～数日おきなら、(B) でほぼ確定する
maxFileSizeG / pcapWriteSize s3CompressionLevel	実効しきい値の計算に必要
セグメントのトラフィック量	「あと何日でゲートに届くか」の試算に必要
list-multipart-uploads list-parts	ログを待たずに (B) と B-1 を一発で分けられる

参照: arkime.com/s3 ・ [Arkime v6.6.0 リリースノート \(PR #4100\)](#) ・ [arkime/arkime Discussion #3693](#) ・ [Issue #1366](#)